

10-AI-DEVELOPMENT-MANUAL.md

Version : V1.0

Statut : ACTIVE

Catégorie : AI Engineering Framework

Projet : Vitala

1. Objectif

AI-DEVELOPMENT-MANUAL.md définit les méthodes obligatoires de développement utilisées par les IA travaillant sur Vitala.

Ce document explique :

- comment analyser une tâche ;
 - comment préparer une implémentation ;
 - comment modifier le code ;
 - comment structurer les fichiers ;
 - comment tester ;
 - comment livrer une fonctionnalité terminée.
-

2. Règle fondamentale

Une IA ne doit jamais commencer par écrire du code.

Le processus obligatoire est :

Comprendre

↓

Analyser

↓

Planifier

↓

Implémenter

↓

Tester

↓

Vérifier

↓

Documenter

↓

Livrer

3. Analyse obligatoire avant développement

Avant toute tâche, l'IA doit identifier :

3.1 Le domaine concerné

Exemples :

- Authentification
- Utilisateur
- Flash
- Scan
- Radar
- Veille
- Mission
- Social
- Trust
- Administration
- IA
- Database
- API

3.2 Les documents concernés

L'IA doit rechercher :

- BUILD correspondant ;
- règles AI ;
- documents Database ;

- documents API ;
 - documents sécurité ;
 - backlog associé.
-

3.3 Les impacts possibles

L'IA doit vérifier :

- fichiers impactés ;
 - données impactées ;
 - API impactées ;
 - utilisateurs impactés ;
 - sécurité impactée.
-

4. Structure générale du projet

L'IA doit respecter l'organisation officielle.

Exemple :

```
src/  
├─ app  
├─ components  
├─ features  
├─ hooks  
├─ services  
├─ lib  
├─ types  
├─ utils  
├─ stores  
└─ tests
```

Les structures exactes doivent toujours suivre BUILD-ARCHITECTURE.md.

5. Règle de séparation des responsabilités

Une IA doit séparer :

Interface

Responsable :

- affichage ;
 - interaction utilisateur.
-

Composants

Responsable :

- éléments UI réutilisables.
-

Hooks

Responsable :

- logique React ;
 - état ;
 - comportement.
-

Services

Responsable :

- communication externe ;
 - logique métier.
-

Types

Responsable :

- modèles ;
 - contrats.
-

Utils

Responsable :

- fonctions génériques.

6. Création d'une nouvelle fonctionnalité

Une nouvelle fonctionnalité doit suivre :

Feature

↓

Types

↓

Services

↓

Hooks

↓

Components

↓

Pages

↓

Tests

↓

Documentation

L'IA ne doit pas mélanger toutes les couches dans un seul fichier.

7. Règles Frontend

L'IA doit :

- utiliser les composants existants ;
 - respecter le Design System ;
 - garder les composants simples ;
 - gérer les états correctement ;
 - prévoir les erreurs ;
 - respecter l'accessibilité.
-

8. Gestion des états

Toute interface doit gérer :

- chargement ;
- succès ;
- erreur ;
- état vide ;
- absence de permission.

Une interface sans gestion d'état complète est considérée incomplète.

9. Règles Backend

Le backend doit respecter :

- séparation logique ;
- sécurité ;
- validation ;
- gestion des erreurs ;
- logs.

L'IA doit éviter :

- logique métier dans l'interface ;
 - duplication ;
 - accès directs non contrôlés.
-

10. Règles Supabase

Toute modification Supabase doit considérer :

Database

- schéma ;
- migrations ;
- relations.

Auth

- utilisateurs ;
- sessions ;
- permissions.

Storage

- fichiers ;
- sécurité ;
- accès.

Edge Functions

- logique serveur ;
- validation ;
- erreurs.

11. Règles Database

L'IA doit toujours :

- créer des migrations ;
- documenter les changements ;
- respecter les conventions PostgreSQL ;
- appliquer RLS.

Interdit :

- modifier directement la production ;
- supprimer une table sans procédure ;
- contourner les permissions.

12. Règles API

Toute API doit définir :

- endpoint ;
- méthode ;
- paramètres ;

- réponse ;
- erreurs ;
- sécurité.

Une API non documentée est interdite.

13. Gestion des erreurs

Toute fonctionnalité doit prévoir :

- erreurs utilisateur ;
- erreurs système ;
- erreurs réseau ;
- erreurs permission.

Les erreurs doivent être :

- compréhensibles ;
 - loggées ;
 - traitées.
-

14. Règles IA et automatisation

Lorsqu'une fonctionnalité utilise l'IA :

L'IA doit définir :

- objectif ;
- entrée ;
- sortie ;
- limites ;
- sécurité ;
- coût.

Une IA sans contrôle est interdite.

15. Tests obligatoires

Selon la fonctionnalité :

Frontend

- tests composants ;
- tests interactions.

Backend

- tests services ;
- tests API.

Database

- tests permissions ;
 - tests RLS.
-

16. Performance

L'IA doit considérer :

- temps de chargement ;
- taille des données ;
- requêtes ;
- cache ;
- consommation mémoire.

Une solution fonctionnelle mais lente n'est pas acceptable.

17. Documentation après développement

Après chaque tâche importante :

Mettre à jour :

- documentation technique ;
 - types ;
 - API ;
 - Database ;
 - changelog si nécessaire.
-

18. Livraison d'une tâche

Une tâche est livrée seulement si :

[] Fonctionnalité créée

[] Architecture respectée

[] Tests réalisés

[] Sécurité vérifiée

[] Documentation mise à jour

[] Aucun problème critique

19. Résultat attendu d'une IA

Une IA travaillant sur Vitala doit produire :

- du code professionnel ;
 - des explications claires ;
 - des modifications contrôlées ;
 - une validation complète.
-

Historique

Version	Modification
V1.0	Création du manuel de développement IA

Fin de AI/10-AI-DEVELOPMENT-MANUAL.md